

Express.js: Introduction, Setup, Routing, Middleware and Advanced Techniques

1. Introduction to Express.js

Express.js is a lightweight and flexible web application framework for **Node.js**. It is mainly used to build web applications and REST APIs. Express simplifies tasks such as routing, handling HTTP requests, managing middleware, and sending responses.

It is widely used because it is simple, fast, and provides developers with the flexibility to structure applications according to their requirements.

2. Features of Express.js

Some important features of Express.js are:

- Simple and easy-to-use framework.
 - Supports routing for different URLs and HTTP methods.
 - Provides middleware for processing requests.
 - Makes REST API development easier.
 - Supports error handling.
 - Can be integrated with databases such as MongoDB and MySQL.
 - Suitable for building scalable web applications.
-

3. Setting Up an Express Application

First, install Node.js and create a project folder. Open the terminal inside the folder and run:

```
npm init -y
```

```
npm install express
```

Create a file named app.js:

```
const express = require("express");
```

```
const app = express();
```

```
const PORT = 3000;
```

```
app.get("/", (req, res) => {  
  res.send("Welcome to Express.js");  
});
```

```
app.listen(PORT, () => {
```

```
    console.log(`Server running on port ${PORT}`);  
  });
```

Run the application using:

```
node app.js
```

The application can then be accessed at:

```
http://localhost:3000
```

4. Routing in Express.js

Routing determines how an application responds to requests made to different URLs.

Express supports common HTTP methods such as:

- GET – retrieve data
- POST – create data
- PUT – update data
- DELETE – remove data

Example:

```
app.get("/users", (req, res) => {  
    res.send("List of users");  
});
```

```
app.post("/users", (req, res) => {  
    res.send("User created");  
});
```

```
app.delete("/users/:id", (req, res) => {  
    res.send(`User ${req.params.id} deleted`);  
});
```

5. Middleware in Express.js

Middleware functions execute between receiving a request and sending a response. They can be used for logging, authentication, data processing, and error handling.

Example:

```
const logger = (req, res, next) => {  
    console.log(`${req.method} ${req.url}`);  
    next();  
};
```

```
    next();  
};
```

```
app.use(logger);
```

The `next()` function passes control to the next middleware or route handler.

Express also provides built-in middleware such as:

```
app.use(express.json());
```

This allows the application to read JSON data sent in requests.

6. Building REST APIs with Express

REST APIs allow different applications to communicate using HTTP methods. Express.js is commonly used for creating REST APIs.

Example:

```
const express = require("express");
```

```
const app = express();
```

```
app.use(express.json());
```

```
let users = [  
  { id: 1, name: "Vedant" },  
  { id: 2, name: "Rahul" }  
];
```

```
app.get("/api/users", (req, res) => {  
  res.json(users);  
});
```

```
app.post("/api/users", (req, res) => {  
  const user = {  
    id: users.length + 1,  
    name: req.body.name  
  };  
});
```

```
    users.push(user);
    res.status(201).json(user);
  });

app.listen(3000, () => {
  console.log("API running on port 3000");
});
```

In this example, GET retrieves users and POST adds a new user.

REST APIs can later be connected to databases to store permanent application data.

7. Error Handling

Express provides mechanisms for handling errors in applications. An error-handling middleware has four parameters:

```
app.use((err, req, res, next) => {
  console.error(err.message);

  res.status(500).json({
    error: "Internal Server Error"
  });
});
```

This prevents application errors from producing unclear responses to users.

8. Advanced Express Techniques

For larger applications, Express can be improved using techniques such as:

Router

Routes can be separated into different files using `express.Router()`.

```
const router = express.Router();

router.get("/users", (req, res) => {
  res.json({ message: "Users list" });
});

app.use("/api", router);
```

Authentication

Authentication middleware can be used to verify users before allowing access to protected routes.

Environment Variables

Sensitive information such as database passwords and API keys should be stored in environment variables instead of directly in the source code.

Example:

```
const PORT = process.env.PORT || 3000;
```

9. Optimizing Express Applications

Express applications can be optimized using several techniques:

- Use asynchronous operations to avoid blocking requests.
- Enable compression for responses.
- Validate incoming data.
- Use caching when appropriate.
- Use efficient database queries.
- Keep middleware and routes organized.
- Avoid storing sensitive information directly in source code.
- Use production logging and error handling.

These practices improve the **performance, security, and scalability** of Express applications.